



Inteligência Artificial | FGA0021 | Turma 01 | 2026.1

Professor: Dr. Fabiano Araujo Soares

Aluno: Bruno Cruz Garcia Rosa

Matrícula: 222006605

Busca Complexa e Algoritmo Genético

1. Introdução

Busca complexa e algoritmo genético se baseiam em algoritmos de busca local, que utilizam estratégias iterativas para melhorar progressivamente soluções candidatas. Diferentemente dos algoritmos clássicos de busca em espaço de estados, a busca local normalmente trabalha apenas com o estado atual e seus vizinhos, sem necessidade de explorar toda a árvore de possibilidades. No entanto, a busca local não garante encontrar a solução global ótima, ou seja, não garante que a solução encontrada é a melhor de todas, mas sim a melhor dada aquelas condições iniciais.

Neste projeto, foi abordado o **Problema do Caixeiro Viajante** (Travelling Salesman Problem – **TSP**), um dos problemas clássicos de otimização combinatória. O problema consiste em determinar a menor rota possível que permita visitar um conjunto de cidades exatamente uma vez e retornar ao ponto de origem.

Para resolução do problema, foram desenvolvidas duas abordagens distintas:

- ❖ **Hill Climbing**, representando algoritmos de busca local;
- ❖ **Algoritmo Genético**, representando algoritmos evolutivos inspirados em seleção natural.

Embora ambos tenham como objetivo minimizar a distância total percorrida, os algoritmos utilizam estratégias completamente diferentes para explorar o espaço de soluções.

Além disso, foi utilizada a biblioteca **Pygame** para a construção da interface gráfica e visualização da exploração dos algoritmos no labirinto, permitindo acompanhar visualmente as rotas entre cidades e a mudança das mesmas. Para execução correta do projeto, é necessária a instalação prévia da biblioteca Pygame através do comando:

- `pip install pygame`

ou:

- `pip install pygame-ce`

2. Definição do Problema

O **Problema do Caixeiro Viajante (TSP)** consiste em encontrar a menor rota possível entre um conjunto de cidades.

No problema, um agente deve:

- iniciar em uma cidade aleatória;
- visitar todas as demais cidades exatamente uma vez;
- retornar à cidade inicial ao final do percurso.

O principal objetivo consiste em minimizar a distância total percorrida durante a rota.

2.1. Complexidade do Problema

O **TSP** é considerado um problema de alta complexidade computacional, pertencente à classe **NP-Hard**. A quantidade de soluções possíveis cresce rapidamente conforme o número de cidades aumenta. Para um conjunto de N cidades, existem aproximadamente $N!$ (N fatorial) rotas possíveis.

Como consequência, métodos exatos tornam-se inviáveis para grandes quantidades de cidades, exigindo o uso de técnicas heurísticas e aproximadas.

Neste projeto, foram utilizadas abordagens heurísticas capazes de encontrar soluções de boa qualidade em tempo viável, mesmo sem garantir a solução ótima global.

2.2. Sobre o Hill Climbing utilizado

O algoritmo Hill Climbing desenvolvido neste projeto utiliza uma abordagem clássica de busca local. A partir de uma solução inicial aleatória, o algoritmo gera pequenas modificações na rota atual através da troca de posições entre cidades. Caso a nova solução apresente menor distância total, ela substitui a solução anterior. O processo é repetido iterativamente até que não sejam encontradas melhorias significativas. A implementação realizada não utiliza variações mais avançadas.

Foi utilizada uma versão simples de Hill Climbing, focada na análise do comportamento básico da busca local e suas limitações, como a possibilidade de ficar presa em ótimos locais.

3. Algoritmos Desenvolvidos

3.1. City

Antes da implementação dos algoritmos de busca local e otimização evolutiva, foi desenvolvido um conjunto de funções auxiliares responsáveis pela geração das cidades, cálculo das distâncias e criação de soluções vizinhas para o problema do TSP.

A função `generate_cities(num_cities)` é responsável por gerar aleatoriamente as cidades utilizadas no problema. Cada cidade é representada por uma coordenada

bidimensional (x, y) dentro da área de visualização do Pygame. O algoritmo gera posições aleatórias respeitando limites mínimos e máximos da tela, evitando que cidades sejam criadas muito próximas das bordas da janela gráfica. Dessa forma, cada execução do projeto produz um conjunto diferente de cidades, permitindo testes variados para os algoritmos de otimização.

A função *distance(city1, city2)* calcula a distância euclidiana entre duas cidades. O cálculo é realizado através da fórmula:

$$d(a, b) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}, \text{ onde:}$$

- x_1 e y_1 representam as coordenadas da primeira cidade;
- x_2 e y_2 representam as coordenadas da segunda cidade.

Essa função é utilizada como base para o cálculo do custo total das rotas.

A função *route_distance(route)* calcula a distância total percorrida em uma rota completa do TSP. O algoritmo percorre todas as cidades da rota somando as distâncias entre cidades consecutivas. Ao final, também é adicionada a distância entre a última cidade e a cidade inicial, garantindo o fechamento do ciclo do caixeiro viajante.

Essa função representa diretamente o custo da solução e é utilizada tanto pelo Hill Climbing quanto pelo Algoritmo Genético para avaliar a qualidade das rotas geradas. Quanto menor o valor retornado por *route_distance()*, melhor é considerada a solução encontrada.

A função *generate_neighbor(route)* é responsável por gerar soluções vizinhas para os algoritmos de otimização. Inicialmente, o algoritmo cria uma cópia da rota atual. Em seguida, duas posições aleatórias da rota são selecionadas, realizando uma troca (SWAP) entre as cidades dessas posições. Essa pequena modificação gera uma nova solução semelhante à anterior, porém com uma ordem de visitação diferente.

3.2. Hill Climbing

O algoritmo Hill Climbing foi implementado através da função *hill_climbing(initial_route, max_iterations)*, sendo utilizado como abordagem de busca local para resolução do Problema do Caixeiro Viajante. O algoritmo inicia com uma solução inicial aleatória (*initial_route*), representando uma possível rota entre as cidades do problema.

Inicialmente, a distância total da rota é calculada utilizando a função *route_distance()*. Em seguida, o algoritmo inicia um processo iterativo de busca por melhorias locais, e cada iteração:

1. uma nova solução vizinha é gerada através da função *generate_neighbor()*;
2. a distância da nova rota é calculada;

3. caso a nova solução apresente menor custo, ela substitui a rota atual.

O parâmetro `max_iterations` controla a quantidade máxima de iterações realizadas pelo algoritmo, limitando o número de explorações no espaço de estados. O processo continua até que o número máximo de iterações seja atingido ou que não sejam encontradas melhorias relevantes.

Por utilizar apenas informações locais da solução atual, o Hill Climbing possui baixo custo computacional e rápida execução. Entretanto, essa abordagem também apresenta limitações importantes. Como o algoritmo aceita apenas soluções melhores que a atual, ele pode facilmente ficar preso em ótimos locais, impedindo a descoberta da solução global ótima do problema.

3.3. Algoritmo Genético

O Algoritmo Genético foi implementado como uma abordagem evolutiva para resolução do Problema do Caixeiro Viajante, simulando conceitos inspirados em evolução natural, reprodução e seleção genética.

Diferentemente do Hill Climbing, que trabalha apenas com uma única solução atual, o Algoritmo Genético manipula simultaneamente uma população inteira de soluções candidatas. Cada indivíduo da população representa uma rota completa entre as cidades do problema.

A função `create_population(cities, population_size)` é responsável por gerar a população inicial do algoritmo. Inicialmente, o algoritmo cria diversas rotas aleatórias utilizando embaralhamento das cidades através da função `random.shuffle()`. O parâmetro `population_size` define a quantidade de indivíduos presentes em cada geração.

A função `fitness(route)` é utilizada para avaliar a qualidade de cada indivíduo da população. O cálculo é realizado utilizando o inverso da distância total da rota:

$fitness = 1/distância$. Dessa forma quanto menor a rota, maior o fitness.

A função `select_best(population, elite_size)` implementa o processo de seleção dos melhores indivíduos da população atual. Inicialmente, todas as rotas são ordenadas com base no valor de fitness. Em seguida, os indivíduos com maior qualidade são selecionados para compor a elite da geração. O parâmetro `elite_size` define quantos indivíduos serão preservados para geração da próxima população.

A função `crossover(parent1, parent2)` implementa o operador genético de recombinação entre dois indivíduos. O algoritmo seleciona aleatoriamente dois pontos de corte dentro da rota do primeiro pai. O trecho entre esses pontos é copiado diretamente para o descendente. Posteriormente, as cidades restantes são preenchidas

utilizando a ordem presente no segundo pai, evitando repetições e garantindo que todas as cidades apareçam exatamente uma vez na nova rota.

A função *mutate(route, mutation_rate)* é responsável por introduzir diversidade genética na população. Durante o processo de mutação, duas cidades da rota podem trocar de posição aleatoriamente. O parâmetro *mutation_rate* controla a probabilidade da mutação ocorrer.

A função *create_next_generation(population, elite_size, mutation_rate)* é responsável pela construção da próxima geração do algoritmo. Inicialmente, os melhores indivíduos da população atual são preservados através do elitismo.

Em seguida:

- Pais são selecionados aleatoriamente da elite;
- Crossover é aplicado para gerar descendentes;
- Mutações são realizadas nos filhos gerados.

O processo continua até que a nova população atinja o mesmo tamanho da geração anterior.

A função *genetic_algorithm(cities, population_size, elite_size, mutation_rate, generations)* coordena toda a execução do algoritmo evolutivo. Inicialmente, é criada uma população de rotas aleatórias utilizando *create_population()*. A partir disso, o algoritmo executa múltiplas gerações sucessivas de evolução genética. Em cada geração:

- A população é atualizada;
- Novos indivíduos são criados;
- O fitness das soluções é avaliado;
- A melhor rota atual é armazenada.

O parâmetro *generations* define quantas gerações evolutivas serão executadas.

Durante a execução, o algoritmo também utiliza Pygame para a visualização gráfica da melhor solução encontrada em tempo real. A interface exibe:

- rota atual;
- distância total;
- geração atual;
- fitness;
- porcentagem de melhoria em relação à solução inicial.

Ao final da execução, o algoritmo retorna:

- a melhor rota encontrada;
- a menor distância obtida.

3.4. Pygame

Para facilitar a análise visual do comportamento dos algoritmos, foi utilizada a biblioteca Pygame para construção da interface gráfica do projeto.

A visualização gráfica permite acompanhar em tempo real a evolução das rotas geradas pelos algoritmos Hill Climbing e Algoritmo Genético, tornando mais intuitiva a interpretação dos resultados obtidos.

A função *draw_route(screen, route)* é responsável por desenhar visualmente a rota atual entre as cidades. Para conjuntos pequenos de cidades, a função exibe visualmente as distâncias entre cada trecho da rota, facilitando a análise manual das soluções encontradas.

A função *draw_cities(screen, cities)* desenha cada cidade do problema utilizando círculos vermelhos posicionados de acordo com suas coordenadas bidimensionais. Cada cidade representa um ponto que deve ser visitado exatamente uma vez durante a rota do caixeiro viajante.

A função *drawGenetics()* é utilizada durante a execução do Algoritmo Genético.

A função *drawHillClimbing()* realiza a visualização gráfica do algoritmo Hill Climbing.

Ambas funções recebem parâmetros específicos para apresentação visual enquanto o algoritmo é animado.

A função *initialize_screen()* é responsável pela criação da janela gráfica principal do projeto. Essa função centraliza a inicialização da interface gráfica utilizada pelos algoritmos.

3.5. Execução Principal

O arquivo principal do projeto é responsável por configurar os parâmetros dos experimentos e iniciar a execução dos algoritmos. Inicialmente, são definidos parâmetros importantes do problema, como:

- Quantidade de cidades;
- Tamanho da população;
- Quantidade de gerações;
- Taxa de mutação;
- Número máximo de iterações.

Em seguida, as cidades são geradas aleatoriamente através da função *generate_cities()*.

Para o Hill Climbing, uma rota inicial aleatória é criada utilizando o embaralhamento das cidades. Por fim, o algoritmo desejado é executado:

- *genetic_algorithm()* para abordagem evolutiva;
- *hill_climbing()* para busca local.

Durante a execução, os resultados são exibidos graficamente através do Pygame, permitindo acompanhar visualmente o comportamento dos algoritmos e a melhoria progressiva das soluções encontradas.

Para uma melhor visualização, recomenda-se utilizar no máximo 200 cidades em ambos os algoritmos, evitando excesso de informações na tela e reduzindo a poluição visual durante a animação.

No Hill Climbing, recomenda-se utilizar no máximo dez mil iterações, evitando execuções excessivamente longas e tornando mais prática a observação da evolução das soluções.

Já no Algoritmo Genético, o tamanho da população influencia diretamente o desempenho da animação. Populações muito grandes aumentam significativamente a quantidade de cálculos realizados a cada geração, reduzindo a fluidez da visualização gráfica. Dessa forma, recomenda-se utilizar populações de até 300 indivíduos para manter uma execução visual mais rápida e estável.

Além disso, o tempo total da animação no Algoritmo Genético depende diretamente da combinação entre:

- tamanho da população;
- quantidade de gerações.

Quanto maiores esses parâmetros, maior será o custo computacional total da execução. Recomenda-se utilizar no máximo 500 gerações quando forem utilizadas populações maiores.

Essas limitações ocorrem tanto devido à alta complexidade computacional dos algoritmos quando relacionada ao custo adicional da renderização gráfica em tempo real utilizando Pygame. Durante a execução, a interface é constantemente atualizada para desenhar cidades, rotas, textos e métricas dos algoritmos, aumentando significativamente o processamento necessário para manter a animação fluida.

No caso específico do Algoritmo Genético, o custo computacional é ainda mais elevado, pois o algoritmo manipula simultaneamente múltiplas soluções por geração, realizando operações de seleção, crossover, mutação e cálculo de fitness para toda a população.

4. Conclusão

Os projetos desenvolvidos permitiram aplicar na prática conceitos importantes de busca local e algoritmos evolutivos através da resolução do Problema do Caixeiro Viajante.

O algoritmo Hill Climbing apresentou implementação simples, baixo custo computacional e rápida convergência para soluções razoáveis. Entretanto, devido à sua

natureza gulosa, demonstrou limitações relacionadas à presença de ótimos locais, podendo deixar de encontrar soluções globalmente melhores.

Por outro lado, o Algoritmo Genético apresentou maior capacidade de exploração do espaço de soluções, utilizando mecanismos de seleção, crossover e mutação para gerar rotas progressivamente melhores ao longo das gerações. Apesar disso, o algoritmo possui maior custo computacional e maior tempo de execução devido à manipulação simultânea de múltiplos indivíduos e interação com o Pygame.

A utilização do Pygame contribuiu para a análise visual dos algoritmos, permitindo observar em tempo real o comportamento das soluções e a evolução das rotas geradas.

De forma geral, os resultados obtidos demonstraram as vantagens e limitações de cada abordagem, evidenciando como diferentes estratégias de busca podem impactar diretamente a qualidade das soluções e o desempenho computacional do problema.

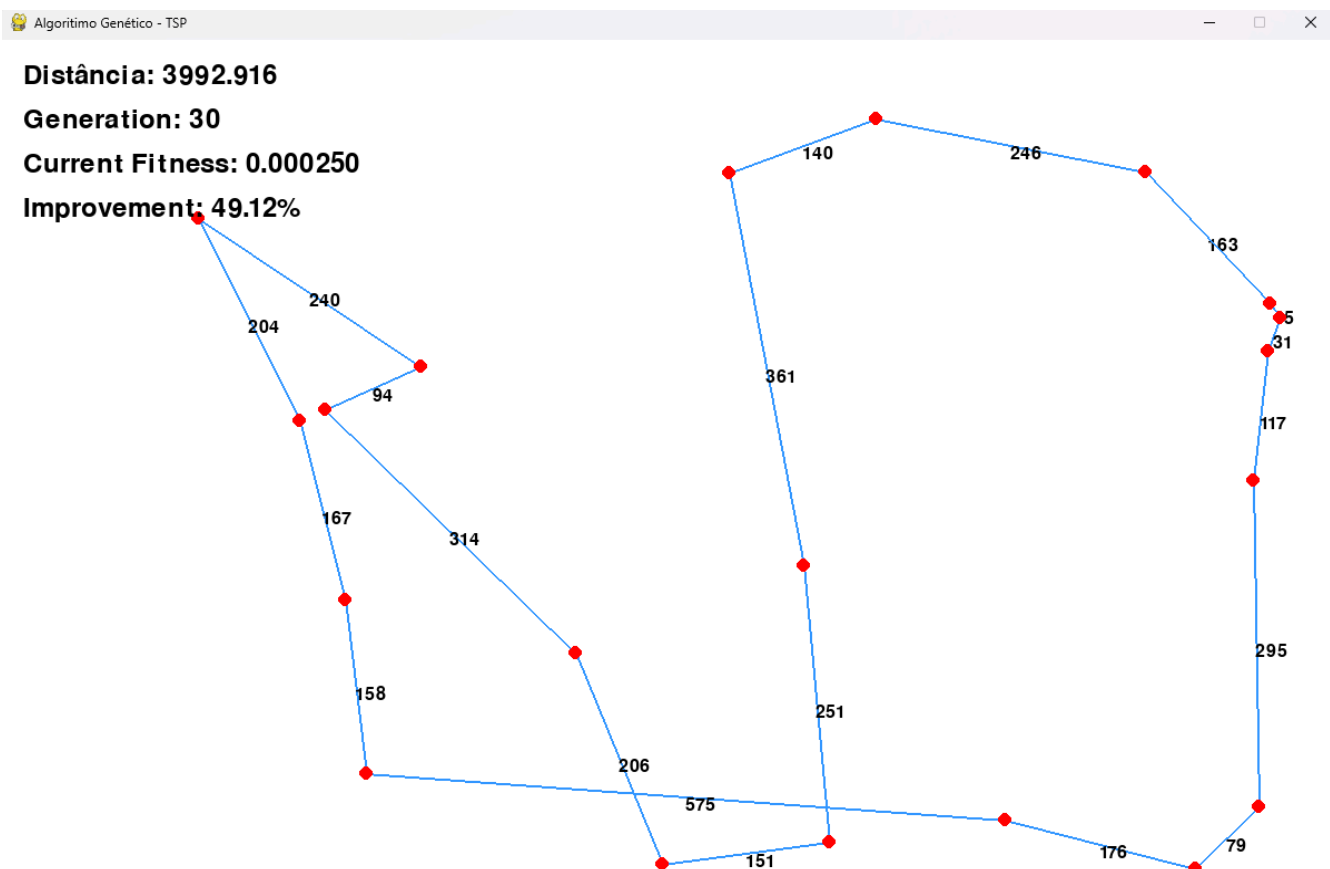


Figura 1 - Resultado visual usando algoritmo genético em um mapa de 20 cidades, tamanho da população=100, tamanho da Elite=5, taxa de mutação=5% e gerações=30. Inicialmente a distância era de 7789,785.